Graded Project – MongoDB & Mongoose

Introduction

Modern web applications rely heavily on databases to manage and query large amounts of structured and semi-structured data. **MongoDB**, a document-oriented NoSQL database, provides flexibility in schema design, while **Mongoose** offers a schema-based modeling solution for Node.js applications.

In this assignment, you will work with an existing Node.js + MongoDB project, define Mongoose models from scratch, and write queries to interact with real database collections.

This project will help you practice:

- Defining Mongoose schemas and models from specifications
- MongoDB CRUD operations
- Query operators (`$gt`, `$gte`, `$in`, `$nin`, `$group`, `$sort`, `$limit`, etc.)
- Working with references between collections
- Aggregation pipelines
- Filtering, projections, and updates
- Async/await patterns in a database context

Project Setup and Execution

1. Download and extract the assignment starter code from the zip file provided on Olympus.

2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension). As you make changes, remember to regularly commit your work.

3. In a terminal, go to the project folder directory `devanswers-backend`.

4. Install Dependencies `npm install`

5. Create a `.env` file in the root of the project with the following content:

   ```
   MONGODB_URI=<your_connection_string>
   ```

6. Populate the database with sample data: `npm run populate`

7. Run the queries : `npm run queries`

> **Important:** You must implement the models before populating the database or running the queries, as both depend on correctly defined schemas (see Problem Statement below).

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

Problem Statement

You are provided with a Node.js project containing sample datasets and scaffolded model files for **User**, **Question**, and **Answer**. Your task is to:

1. **Task 1: Implement the three Mongoose models** (`User`, `Question`, `Answer`) based on the schema specifications in the Data Models section below.
2. **Task 2: Write a collection of 20 MongoDB-Mongoose queries** inside a single file named `queries.js`.

Each query function (`query1` to `query20`) is already created for you. Your goal is to implement the correct logic for each query using Mongoose methods.

File Structure

```
devanswers-backend/
├── .env.example
├── package.json
├── queries.js
├── models/
│   ├── Answer.js
│   ├── Question.js
│   └── User.js
└── scripts/
    ├── populate_db.js
    └── seed-data.js
```

- **.env.example** – Shows the environment variables required to run the project. Copy this to `.env` and fill in your MongoDB connection string.
- **models folder** – Contains scaffolded Mongoose model files. You must implement the schema fields for each model.
- **scripts folder** – Contains `populate_db.js` (the database seeder) and `seed-data.js` (sample datasets for users, questions, and answers).
- **queries.js** – File where you will write all MongoDB + Mongoose queries.

Task 1: Building the Data Models

You must define three Mongoose models based on the specifications below. Each model file in the `models/` folder already has the basic boilerplate (imports, `mongoose.model()`, and export). Your job is to define the schema fields.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

User Model (`models/User.js`)

Field	Type	Required	Additional Info
`name`	String	Yes	—
`email`	String	Yes	Unique: `true`
`password`	String	Yes	—
`createdAt`	Date	No	Default: `Date.now`

Question Model (`models/Question.js`)

Field	Type	Required	Additional Info
`title`	String	Yes	—
`description`	String	Yes	—
`tags`	Array of String	No	Default: `[]`
`upvotes`	Array of ObjectId	No	Each element is a ref to `User`. Default: `[]`
`downvotes`	Array of ObjectId	No	Each element is a ref to `User`. Default: `[]`
`voteCount`	Number	No	Default: `0`
`views`	Number	No	Default: `0`
`author`	ObjectId	Yes	Ref: `User`
`createdAt`	Date	No	Default: `Date.now`

Answer Model (`models/Answer.js`)

Field	Type	Required	Additional Info
`questionId`	ObjectId	Yes	Ref: `Question`
`answerText`	String	Yes	—
`author`	ObjectId	Yes	Ref: `User`
`upvotes`	Array of ObjectId	No	Each element is a ref to `User`. Default: `[]`
`downvotes`	Array of ObjectId	No	Each element is a ref to `User`. Default: `[]`
`voteCount`	Number	No	Default: `0`
`createdAt`	Date	No	Default: `Date.now`

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.

Task 2: Functional Requirements

Below are the 20 queries you must complete. All queries should use **Mongoose** and **async/await**.

1. Create a user with name Robin, email robin@example.com, password hashed_password_7, and createdAt set to 2025-06-25T10:15:00Z
2. Fetch the user with email alice@example.com.
3. Fetch question with the title 'How can I improve the performance of a react app?'
4. Find all questions tagged with 'javascript'.
5. Retrieve all questions posted after April 1, 2023.
6. Find all questions tagged with javascript or react.
7. Find all the distinct tags used in questions.
8. Retrieve all questions with at least 50 views.
9. List all answers with a vote count of 0.
10. Retrieve all answers with a voteCount greater than 0.
11. Retrieve all users whose account was created between January 1, 2023 (inclusive) and May 1, 2023 (exclusive).
12. Fetch the answer text and author id of all answers for the question 'How do I set up routing with react router v6?'
13. Find all users who have not posted any answers.
14. Find the top two questions with the highest voteCount.
15. Retrieve the ids of all users who have posted answers, along with the number of answers they have posted.
16. Identify the top two users who posted the most answers.
17. Update the tags of the question 'Why is my async function returning a promise instead of the actual value?' to ['javascript', 'async']
18. Update the name of the user with email 'alice@example.com' to 'Alice Smith'.
19. Delete the user with email 'jhonny@example.com'.
20. Delete all answers of the user with email 'alice@example.com'.

Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution is prohibited.